

Lecture 2:
Object Oriented Programming I

Procedural vs. Object-Oriented Programming

- The unit in procedural programming is *function*, and unit in object-oriented programming is *class*
- Procedural programming concentrates on creating functions, while object-oriented programming starts from isolating the classes, and then look for the methods inside them.
- Procedural programming separates the data of the program from the operations that manipulate the data, while object-oriented programming focus on both of them

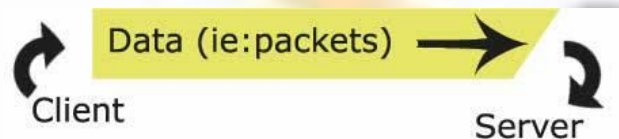


figure1: procedural

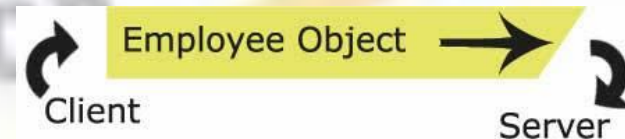


figure2: object-oriented

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called functions .	In object-oriented programming, the program is divided into small parts called objects .
Procedural programming follows a top-down approach .	Object-oriented programming follows a bottom-up approach .
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is less secure .	Object-oriented programming provides data hiding so it is more secure .
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.

In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the <i>unreal world</i> .	Object-oriented programming is based on the <i>real world</i> .
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	Examples: C++, Java, Python, C#, etc.

Concept of Class and Object

- “Class” refers to a blueprint. It defines the variables and methods the objects support
- “Object” is an instance of a class. Each object has a class which defines its data and behavior

Class Members

- **A class can have three kinds of members:**
 - ***fields***: data variables which determine the status of the class or an object
 - ***methods***: executable code of the class built from statements. It allows us to manipulate/change the status of an object or access the value of the data member
 - ***nested classes and nested interfaces***

The slide features a background watermark of the PUACP logo, which includes a shield with a book and the text 'PUACP'. At the top, there are five circles: three solid purple and two hollow purple. The title 'Sample class' is written in a bold, italicized, black serif font and is underlined.

Sample class

```
class Pencil {  
    public String color = "red";  
    public int length;  
    public float diameter;  
  
    public static long nextID = 0;  
  
    public void setColor (String newColor) {  
        color = newColor;  
    }  
}
```

Fields – Declaration

- **Field Declaration**

- a type name followed by the field name, and optionally an initialization clause
- primitive data type vs. Object reference
 - boolean, char, byte, short, int, long, float, double
- field declarations can be preceded by different modifiers
 - access control modifiers
 - static
 - final

More about field modifiers (1)

- Access control modifiers
 - *private*: private members are accessible only in the class itself
 - *package*: package members are accessible in classes in the same package and the class itself
 - *protected*: protected members are accessible in classes in the same package, in subclasses of the class, and in the class itself
 - *public*: public members are accessible anywhere the class is accessible